

Pricing Dinámico Retail — Product Requirements Document (PRD)

Aplicación web de pricing dinámico, elasticidad precio-demanda y proyección de ventas para el retail mexicano. Desarrollada en Python / Streamlit, orientada a analistas de precios y equipos comerciales de Office Max México.

Tabla de contenidos

1. Descripción general
2. Componentes de la aplicación
3. Modelo de inteligencia artificial
4. Lógica de la aplicación
5. Estructura del repositorio
6. Requisitos técnicos
7. Cómo correr localmente

1. Descripción general

La aplicación permite a Office Max México tomar decisiones de precio basadas en datos, combinando limpieza de ventas históricas, cruce con nivel socioeconómico (NSE) por municipio, cálculo de elasticidad precio-demanda por SKU y segmento NSE, simulación de escenarios de cambio de precio (–20 % a +20 %) y promociones (2x1, 3x2, segundo al 50 %), proyección de demanda futura y generación de recomendaciones ejecutivas priorizadas. Todo el flujo ocurre dentro de una interfaz web sin necesidad de escribir código.

El sistema está construido sobre una **arquitectura de pipeline en capas**: cada etapa produce una tabla interna nombrada (`ventas_nse`, `elasticidades_periodo`, `pricing_historico_escenarios`, `demanda_base_futura`, `pricing_futuro_escenarios`, `recomendaciones_sku`) que alimenta a la siguiente, de modo que un cambio de filtro visual nunca recalcula los modelos, solo filtra tablas ya materializadas.

2. Componentes de la aplicación

La aplicación se compone de **seis vistas de usuario** (`app.py`), un **motor de procesamiento de nueve módulos** (`modules/`) y una **capa de datos precargados** (`data/`). El enrutador de `main()` ejecuta **una sola vista por interacción** (`if vista.startswith("1.") ... return`), de modo que la app nunca corre cálculos de vistas que el usuario no está viendo. Cada cálculo pesado se dispara con un botón explícito y se guarda en **doble caché**: el decorador `@st.cache_data` de Streamlit y un diccionario manual `st.session_state.manual_cache` indexado por la firma de los archivos cargados.

A continuación se describe cada componente, qué hace y el valor que aporta.

2.1 Componente transversal — Panel lateral y motor de estado (`render_sidebar`, `init_state`)

Qué hace:

- **Navegación por vistas:** un `st.sidebar.radio` con seis ítems numerados; el ítem activo se resalta en amarillo mediante CSS personalizado. Es la única fuente de verdad de qué vista se renderiza.
- **Carga de tres archivos:** (A) base de ventas obligatoria, (B) base de promociones opcional, (C) configuración NSE. Acepta CSV, Excel (`.xlsx/.xls`) y Parquet.
- **Lectura diferida:** los archivos solo se leen al presionar «Procesar / actualizar datos», nunca en cada *rerun*. La firma de cada archivo (`get_uploaded_file_signature`: nombre + tamaño + hash blake2b del inicio/fin) detecta si la base cambió, e invalida los cachés solo cuando es necesario.
- **Estado de sesión:** `init_state()` inicializa ~50 claves de `st.session_state` (banderas `*_ready`, tablas intermedias, firmas, selección de método/horizonte futuro). `reset_model_results()` limpia todos los derivados cuando cambia la base, garantizando que nunca se mezclen resultados de bases distintas.
- **Botón «Limpiar caché»:** ejecuta `st.cache_data.clear()` y reinicia el estado sin recargar la página.

Valor para el usuario: centraliza la entrada de datos y la navegación, y garantiza rendimiento: gracias a la lectura diferida y la doble caché, mover un filtro o cambiar de vista responde de forma inmediata sin reprocesar 50 000+ filas.

2.2 Vista 1 — Carga y diagnóstico de datos (Data Quality Engine + NSE Configuration Engine)

Implementada en `render_quality_view`, `process_quality_pipeline` y los módulos `modules/utils.py` y `modules/quality.py`.

Qué hace:

- **Lectura robusta (`read_uploaded_file`):** detecta automáticamente encoding (UTF-8, UTF-8-SIG, Latin-1, CP1252), separador (, o ;) y motor (PyArrow rápido → engine C de pandas como respaldo). Para CSV/Excel lee **solo las columnas necesarias** (`COLUMNAS_LECTURA_VENTAS`) para acelerar bases grandes; Parquet se lee de forma columnar.
- **Normalización de columnas (`normalize_column_names`):** estandariza encabezados y mapea decenas de *alias* de negocio (p. ej. `fecha`, `unidades`, `producto`, `sku`) a nombres canónicos (`tran_date`, `qty`, `prod_nbr`) usando tokens sin acentos ni espacios. Esto evita el clásico error `KeyError: 'SKU'` cuando el archivo del usuario usa otros nombres.
- **Limpieza de texto (`clean_text_columns`):** elimina espacios dobles e iniciales/finales en todas las columnas de texto, con prioridad para `store_nm`.
- **Limpieza de ventas (`clean_sales_data`):** parseo de fechas con formato mexicano `dd/mm/YYYY` y respaldos (con hora, ISO, `dayfirst`); conversión numérica que tolera símbolos \$ y separadores de miles; eliminación de duplicados, nulos en columnas críticas, `qty ≤ 0`, `net_sale ≤ 0`, precios `≤ 0`, costos negativos e infinitos. Calcula columnas derivadas: `precio_unitario = net_sale / qty`, `costo_unitario`, `margen_unitario`, `margen_total`, `precio_base`, `ingreso_base`.
- **Variables de periodo (`add_period_variables`):** crea `mes`, `año`, `trimestre`, `semestre`, `periodo_mensual`, `periodo_trimestral`, `periodo_semestral`, `periodo_anual`.
- **Cruce NSE en dos pasos (`merge_sales_with_nse`):** 1. **Catálogo geográfico** (`catalogo_ubica_geo.csv`): traduce la llave de texto `key` del registro a `id_municipio` (`ubica_geo`). 2. **Base INEGI hogares** (`hogares_INEGI.csv`): por cada municipio calcula el NSE dominante (**moda estadística** de `est_socio`, normalizado a `bajo/medio bajo/medio alto/alto` por `normalizar_categoria_est_socio`) y lo une a las ventas como `categoria_est_socio`. Los registros sin cruce se marcan `NSE_no_asignado` (nunca se eliminan). Se agrega trazabilidad: `fuentes_nse`, `nse_asignado`, `nse_match_status` (`match_default/match_personalizado/fallback_default/sin_match`).

- **NSE Configuration Engine:** la app funciona aunque el usuario solo suba ventas, porque las bases NSE vienen precargadas. El usuario puede subir una base NSE personalizada; `validate_custom_nse` valida que no esté vacía, que tenga columna NSE válida, claves de cruce compatibles con las ventas, sin nulos ni duplicados conflictivos, y valores NSE válidos. Si falla, la app **no se rompe**: usa la default como *fallback* y lo registra en `estado_validacion_nse`.
- **Semáforo de calidad (`calculate_quality_diagnosis`):** clasifica la base en [ROJO] Rojo / [AMARILLO] Amarillo / [VERDE] Verde según pocas filas/SKUs, SKUs sin observaciones o variación de precio suficientes, porcentaje de registros removidos (umbrales 25 % amarillo / 50 % rojo) y coeficiente de variación alto.
- **Diagnóstico consolidado (`build_quality_diagnostics`):** tabla interna `diagnostico_calidad` con registros iniciales/finales/eliminados, nulos por columna, duplicados, SKUs/tiendas/categorías/departamentos únicos, periodos disponibles, varianza de precio/unidades por SKU, SKUs con datos suficientes vs. insuficientes, y métricas del cruce NSE (`fuentes_nse_usada`, `porcentaje_match_nse`, `registros_sin_match_nse`, `advertencias_nse`).
- **Análisis ML histórico opcional (`build_historical_sales_ml_summary`):** entrena Regresión Logística + Random Forest para explicar qué variables separan los meses de venta alta vs. baja (ver §3.4).

Valor para el usuario: garantiza que todo el análisis posterior parte de datos confiables y trazables. El semáforo le dice al analista, antes de cualquier modelo, si la base es apta para pricing automático, usable con restricciones, o no confiable.

2.3 Vista 2 — Elasticidad (Elasticity Engine)

Implementada en `render_elasticity_view` y `modules/elasticity.py` (`calculate_elasticity`, `calculate_elasticidades_periodo`).

Qué hace:

- **Cálculo multi-periodo:** estima elasticidad en seis granularidades (`PERIODOS_ELASTICIDAD`): `mensual`, `trimestral`, `semestral`, `anual`, `global_sku` y `categoria_departamento`.
- **Estratificación NSE:** dentro de cada SKU × periodo calcula una elasticidad **por segmento NSE**. Si un segmento no tiene datos suficientes, hace *fallback* a la elasticidad del SKU completo (`_apply_nse_fallback_vectorized`).
- **Fallback en cascada:** SKU × NSE → SKU completo → categoría del mismo periodo → departamento del mismo periodo; cada paso se aplica de forma **vectorizada** (sin reentrenar modelos fila por fila).
- **Evaluación de confianza (`_evaluate_confidence_frame`):** clasifica cada estimación en `Alta/Media/Baja/No usable` según R², p-value, número de observaciones del modelo y precios distintos (ver §3.1). Solo `Media/Alta` se usan en pricing.
- **Visualizaciones:** resumen de disponibilidad (`periodo_tipo` × confianza), KPIs filtrados, desglose y gráfico de barras de elasticidad por NSE, tabla completa, serie de tiempo de demanda semanal (con/sin promoción), curva precio-demanda log-log (`build_elasticity_curve_data`) y mapa geográfico de México coloreado por intensidad de elasticidad por estado.

Valor para el usuario: cuantifica con precisión cuánto cambia la demanda ante un cambio de precio, segmentado por tipo de cliente (NSE) y por periodo. Es el insumo fundamental de toda simulación de pricing.

2.4 Vista 3 — Pricing histórico / backtesting (Historical Pricing Simulator)

Implementada en `render_historical_pricing_view` y `modules/historical_pricing.py` (`build_pricing_historico_escenarios`).

Qué hace:

- **Backtesting de 12 escenarios:** para cada SKU × NSE × periodo histórico simula 9 cambios de precio (−20 %...+20 %) + 3 promociones (2x1, 3x2, 2do al 50 %).
- **Fórmula de elasticidad constante:** $\text{unidades_simuladas} = \text{unidades_reales} \times \exp(e \times \ln(1 + \Delta p))$ para cambios simples; $\text{unidades_reales} \times (1 + e \times \Delta p)$ para promociones. Calcula **precio_efectivo**, **ingreso_simulado**, **margen_simulado** y sus variaciones contra lo realmente observado.
- **Mejor escenario histórico:** marca por grupo el escenario que maximiza margen (o ingreso si no hay costo), excluyendo promociones de riesgo alto.
- **Recomendaciones vectorizadas (`_assign_recommendations_vectorized`):** clasifica cada fila en **Mejor escenario histórico**, **Escenario viable**, **Mantener como referencia**, **No preferente** o **No recomendar**.
- **Filtros dependientes en cascada** (Departamento → Categoría → Periodo → SKU → NSE → tipo de elasticidad → escenario) y gráficas real vs. simulado.
- **Análisis de promociones (`modules/promotions.py`):** si se cargó base de promociones, grafica la demanda semanal antes/durante/después de cada promoción.

Valor para el usuario: responde "¿qué habría pasado con ingresos y márgenes si hubiéramos cambiado el precio en un periodo pasado?". Sirve como evidencia cuantitativa para validar decisiones y negociar con proveedores.

2.5 Vista 4 — Pricing futuro (Demand Forecast Engine + Future Pricing Simulator)

Implementada en `render_future_pricing_view`, `modules/demand_forecast.py` y `modules/future_pricing.py`.

Qué hace:

- **Paso 1 — Demanda base futura (`build_demanda_base_futura`):** proyecta unidades al precio actual para horizontes de **1 mes** y **3 meses**, mediante promedio ponderado de seis ventanas (últimos 3/6/12/24 meses, mismo mes histórico, mismo trimestre histórico). El cálculo es totalmente vectorizado (`_compute_all_components_batch`: un *pivot* + medias móviles para todos los SKUs a la vez). Métodos seleccionables: *Automático recomendado*, *Reciente*, *Estacional*, *Histórico amplio*, *Manual avanzado*. Las ventanas faltantes redistribuyen su peso entre las disponibles y bajan la confianza.
- **Paso 2 — Precios base (`_build_price_base`):** precio actual/lista/costo por SKU desde el último mes con datos (una fila por SKU, para acelerar el caché).
- **Paso 3 — Simulación futura (`build_pricing_futuro_escenarios`):** aplica los mismos 12 escenarios con fórmula lineal $\text{unidades_simuladas} = \text{demanda_base} \times (1 + e \times \Delta p)$. Combina **confianza final = $\min(\text{confianza_elasticidad}, \text{confianza_demanda})$** y evalúa riesgo + recomendación de forma vectorizada.
- **UI:** selector de horizonte (1 mes / 3 meses / Ambos), método de proyección, mes de inicio de proyección y, en modo Manual, casillas por ventana. Filtros NSE al final, tabla de mejor escenario por SKU × NSE, gráficas y tablas de confianza/riesgo.

Valor para el usuario: permite decidir el precio del próximo mes/trimestre con base cuantitativa **antes** de implementarlo, separando explícitamente la *proyección de demanda* (qué venderé) de la *elasticidad* (cómo reacciona la demanda al precio).

2.6 Vista 5 — Recomendaciones ejecutivas (Recommendation Engine)

Implementada en `render_recommendations_view` y `modules/recommendations.py` (`generar_recomendaciones`).

Qué hace:

- **Ranking por SKU × horizonte:** una fila por combinación, ordenada por ingreso esperado.
- **Motor híbrido de tres pasos:** (1) reglas de exclusión → **No recomendar**; (2) selección del mejor escenario válido (margen si hay costo, ingreso si no); (3) clasificación en dos niveles: **categoría_recomendacion** (Subir precio / Bajar precio o promover / Mantener precio / No recomendar) y **estrategia_especifica** (p. ej. *Subir precio 10 %, 2x1*).
- **Explicabilidad:** cada recomendación lleva una razón en español que cita la elasticidad, la demanda y el impacto en margen/ingreso.
- **KPIs y filtros:** SKUs con acción, ingreso/margen esperado total, % de SKUs con acción; filtros por categoría, departamento, NSE, horizonte, recomendación y confianza.

Valor para el usuario: convierte decenas de miles de simulaciones en un listado priorizado y accionable que el equipo comercial puede ejecutar directamente, con justificación defendible ante stakeholders.

2.7 Vista 6 — Exportables

Implementada en `render_exportables_view`.

Qué hace: descarga en CSV válido (UTF-8 con BOM, columnas con nombres claros, objetos complejos serializados a JSON-string) las siete tablas internas: `diagnostico_calidad`, `ventas_limpias`, `elasticidades_periodo`, `pricing_historico_escenarios`, `demanda_base_futura`, `pricing_futuro_escenarios`, `recomendaciones_sku`. Cada botón se desactiva si la tabla aún no se ha generado.

Valor para el usuario: integra los resultados con ERP, BI o reportes en Excel/PowerPoint, conservando trazabilidad NSE en cada exportable.

3. Modelo de inteligencia artificial

La aplicación combina **tres motores analíticos**. El núcleo (§3.1) es un modelo econométrico de elasticidad; los demás lo apoyan (pronóstico §3.2, reglas §3.3) y un modelo ML supervisado se usa solo como diagnóstico (§3.4).

3.1 Modelo principal — Regresión OLS log-log (elasticidad precio-demanda)

¿Qué modelo se usa?

El núcleo es una **regresión lineal por Mínimos Cuadrados Ordinarios (OLS) en escala logarítmica** (modelo log-log o de elasticidad constante):

$$\log(Q) = \alpha + \beta \cdot \log(P)$$

donde **Q** = demanda diaria agregada por nivel de precio, **P** = precio unitario del día, **α** = intercepto y **β** = **elasticidad precio-demanda** (el parámetro de interés). En el modelo log-log, **β** se interpreta directamente: un aumento de 1 % en el precio produce un cambio de **β %** en la demanda.

¿Qué lo diferencia de otros modelos?

Característica	OLS log-log	Alternativas (XGBoost, redes, sklearn LinearRegression)
----------------	-------------	---

Interpretación	β es la elasticidad directa; económicamente significativo	Modelos ML no entregan elasticidades directas
Granularidad	Un modelo por SKU × NSE × periodo → políticas diferenciadas	Modelos globales mezclan SKUs y pierden especificidad
Eficiencia	Fórmula cerrada vectorizada ($\beta = S_{xy}/S_{xx}$) estima cientos de modelos en milisegundos	<code>sklearn</code> exige <code>.fit()</code> por grupo
Estadísticos	Entrega R^2 , p-value (t de Student vía <code>scipy.stats</code>), error estándar	ML no produce p-values nativos
Robustez con pocos datos	Válido desde 3 observaciones agregadas con advertencias de confianza	ML necesita decenas de observaciones por segmento
Causalidad económica	Fundamento teórico: relación log-lineal precio-demanda; valida el signo	"Caja negra" no garantiza signo coherente

¿Cómo se entrena/implementa?

Capa 1 — Vectorizada (ruta de producción, todos los grupos a la vez) — `_estimate_loglog_grouped`:

Para cada combinación SKU × NSE × periodo se agregan las ventas por día × precio (`_build_model_rows`) y se calculan las sumas estadísticas con un único `groupby`:

```
agg = m.groupby(group_cols).agg(
    Observaciones_Modelo=("log_qty", "size"),
    sum_x=("log_precio", "sum"), sum_y=("log_qty", "sum"),
    sum_x2=("x2", "sum"), sum_y2=("y2", "sum"),
    sum_xy=("xy", "sum"),
)
Sxx = sum_x2 - sum_x**2 / n
Sxy = sum_xy - sum_x*sum_y / n
beta = Sxy / Sxx # elasticidad (fórmula cerrada OLS)
alfa = sum_y/n - beta * sum_x/n
R2 = Sxy**2 / (Sxx * Syy)
# p-value: t = beta / EE(beta) con (n-2) g.l. → 2*sf(|t|) (scipy.stats.t)
```

Esto estima todas las elasticidades por SKU × NSE × periodo en una sola pasada. Casos límite controlados: cantidad agregada constante → elasticidad ≈ 0 ; varianza de precio insuficiente → no estimable; infinitos/NaN → descartados.

Capa 2 — `statsmodels.api.OLS` — `estimar_elasticidad_loglog`:

Implementación de referencia con p-values y diagnósticos exactos; sirve de verificación y para grupos pequeños. Comparte exactamente los mismos umbrales (`MIN_OBSERVACIONES = 3`, `MIN_PRECIOS_DISTINTOS = 2`).

¿Qué tan confiables son sus resultados?

La confiabilidad se evalúa en cuatro dimensiones (`_evaluate_confidence_frame`, valores reales de `config.py`):

Criterio	"Alta"	"Media"	"Baja" / "No usable"
R^2	≥ 0.50	0.15 – 0.49	< 0.15 o nulo

p-value	—	≤ 0.20	$> 0.20 \rightarrow$ Baja
Observaciones del modelo	≥ 10 (y ≥ 15 datos crudos)	≥ 5	$< 3 \rightarrow$ No usable
Precios distintos	≥ 4	≥ 3	$< 3 \rightarrow$ No usable
Signo económico	Negativo y ≥ -5	Negativo	Positivo, cero o $< -10 \rightarrow$ inestable

Solo las elasticidades **Media** o **Alta** se usan en las simulaciones; las **Baja/No usable** activan el *fallback* en cascada. La elasticidad además se **acota** a `[-5, 0]` (`ELASTICIDAD_CAP_MIN/MAX`) antes de simular, para impedir que valores extremos generen proyecciones absurdas.

3.2 Motor de pronóstico de demanda — ponderación de ventanas temporales

¿Qué modelo se usa?

Para proyectar la demanda base futura (sin cambio de precio) se usa un **modelo de promedio ponderado de ventanas históricas**, no paramétrico y totalmente interpretable:

$$\text{demanda_base} = \sum (\text{peso_i} \times \text{promedio_ventana_i})$$

Con seis ventanas: últimos 3/6/12/24 meses, mismo mes histórico (estacionalidad mensual) y mismo trimestre histórico (estacionalidad trimestral). Los pesos por defecto (`DEMANDA_FUTURA_PESOS_DEFAULT`) son: para 1 mes $0.50 \cdot \text{últ.3m} + 0.30 \cdot \text{últ.12m} + 0.20 \cdot \text{mismo mes}$; para 3 meses $0.40 \cdot \text{últ.6m} + 0.30 \cdot \text{últ.24m} + 0.30 \cdot \text{mismo trimestre}$.

Métodos disponibles y por qué no es un modelo "caja negra"

Método	Ventanas	Cuándo usarlo
Automático recomendado (default)	Combina recientes + estacionales según disponibilidad	Caso general
Reciente	Últimos 3 m (1 mes) / 6 m (3 meses)	Productos en tendencia
Estacional	Mismo mes / mismo trimestre histórico	Fuerte estacionalidad
Histórico amplio	Últimos 12 m / 24 m	Productos estables
Manual avanzado	Pesos definidos por el analista	Conocimiento experto

Si una ventana no alcanza el mínimo de meses (`DEMANDA_FUTURA_MIN_MESES_VENTANA`), su peso se **redistribuye proporcionalmente** (`_redistribute_weights`) y se baja la confianza. Los pesos son **configurables** sin tocar el motor, dejando preparado un futuro ajuste por backtesting.

Confianza del pronóstico (`_classify_confidence`)

- **Alta:** hay historia reciente y estacional, sin ventanas faltantes y baja volatilidad ($CV < 1.50$).
- **Media:** solo historia reciente, poca estacionalidad.
- **Baja:** pocos datos recientes o demanda muy volátil ($CV \geq \text{DEMANDA_FUTURA_VOLATILIDAD_CV_ALTA} = 1.50$).

- **No usable:** ninguna ventana tiene datos suficientes.

3.3 Motor de recomendaciones — sistema experto basado en reglas

¿Qué modelo se usa?

Un **sistema experto de reglas de negocio sobre la simulación financiera** (`generar_recomendaciones`), no un clasificador entrenado. Combina elasticidad, confianza, demanda base, e impacto en ingreso/margen:

```
# Reglas de exclusión → "No recomendar"
elasticidad NaN/∞/≥0/≈0, confianza_elasticidad o confianza_demanda "No usable",
precio_actual ≤ 0, demanda_base ≤ 0 → No recomendar

# Selección del mejor escenario entre los que cumplen guardrails
si hay costo: maximizar margen_simulado (descartando margen<0 y caídas >80% de volumen)
si no hay costo: maximizar ingreso_simulado (y marcar que no se evaluó margen)

# Clasificación de la mejor opción
mejor con Δp>0 → "Subir precio" (demanda inelástica)
mejor con Δp<0 o promo → "Bajar precio / promover" (demanda elástica)
ninguna supera al escenario base → "Mantener precio"
```

¿Por qué reglas en lugar de ML supervisado?

Garantiza **explicabilidad total**: cada recomendación lleva una razón en español comunicable a stakeholders. Un clasificador ML requeriría etiquetas históricas de "buenas/malas decisiones de precio" que **no existen** en los datos; entrenarlo sobre etiquetas generadas por reglas solo reproduciría las reglas con menos transparencia (*target leakage*).

3.4 Modelo ML de apoyo — Regresión Logística + Random Forest (diagnóstico)

`modules/historical_ml.py` entrena, dentro de un **Pipeline** de scikit-learn (imputación + one-hot + escalado), una **Regresión Logística** y un **Random Forest** para clasificar meses-SKU de venta alta vs. baja y revelar drivers históricos (precio, mes, trimestre, promoción, departamento, NSE, estado).

- **Buenas prácticas:** `train_test_split` (25 % test, estratificado), `class_weight="balanced"`, `max_depth/min_samples_leaf` para evitar sobreajuste; métricas de **accuracy, balanced accuracy y ROC-AUC** sobre el conjunto de prueba; *feature importance* mostrado en la vista 1.
- **Rol acotado:** este modelo es **diagnóstico**; **nunca decide** la recomendación de precio. El motor de recomendaciones lo deja desactivado por defecto (`usar_random_forest = False`) precisamente para evitar *target leakage*. Si se reactivara, solo aportaría `probabilidad_exito/riesgo`, jamás `categoria_recomendacion`.

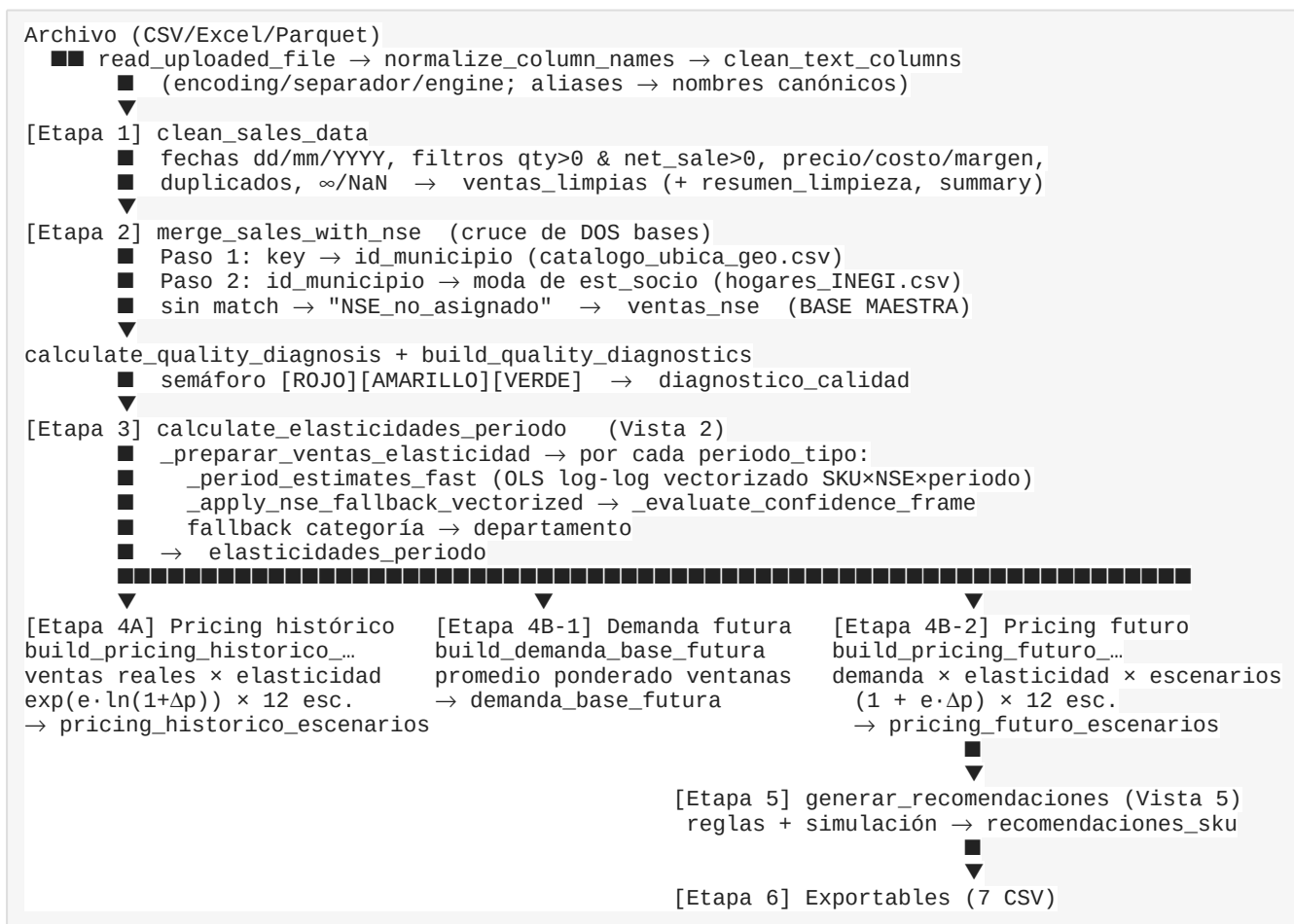
4. Lógica de la aplicación

4.1 Razonamiento detrás de la solución

El diseño responde a tres restricciones del retail mexicano de papelería:

1. **Heterogeneidad por NSE:** clientes de NSE **alto** y **bajo** reaccionan distinto al precio. Una elasticidad única por SKU mezclaría señales contradictorias; por eso se estratifica por NSE y se permiten políticas diferenciadas.
2. **Variabilidad temporal:** la elasticidad en temporada alta difiere de la temporada baja. El modelo multi-periodo (mensual→anual) captura estas variaciones.

- Además, se separa conceptualmente la **elasticidad** (cómo reacciona la demanda al precio) de la **proyección de demanda** (cuántas unidades se venderán), porque mezclarlas produce recomendaciones engañosas.



SI todos son "No recomendar" → omitir el SKU (no se fuerza una recomendación)

Decisión 3 — ¿qué confianza tiene la simulación futura?

confianza_final = mínimo(confianza_elasticidad, confianza_demanda)
Alta+Alta→Alta · Alta+Media→Media · Media+Baja→Baja · cualquiera+No usable→No usable

Decisión 4 — guardrails de promociones (evaluar_riesgo_promocion): una promoción se marca **riesgo Alto** (y no se recomienda) si la elasticidad es ≥ 0/NaN, la demanda base < 5 unidades, el costo ≥ precio efectivo, el margen simulado < 0, o la confianza de elasticidad/demanda es baja.

4.4 Optimizaciones de rendimiento y memoria

Optimización	Impacto
Una vista por rerun + botones explícitos + doble caché	No recomputa al cambiar de vista o mover filtros
OLS vectorizado (fórmula cerrada en groupby)	Estima cientos de modelos en ms, no s
Pre-limpieza de ventas una sola vez antes del bucle de periodos	−60 % tiempo en pricing histórico
Dedup de candidatos antes del join cartesiano ×12	Reduce el volumen del paso más costoso
Recomendaciones/riesgo vectorizados (numpy masking)	Reemplaza apply(axis=1) fila por fila
_compute_all_components_batch (pivot + medias)	Demanda futura sin bucle por SKU
float64→float32 + object→category en tablas de salida	−50 % a −70 % de RAM
_slim_elasticidades (10 columnas) y price_base (1 fila/SKU)	Hashing de caché de Streamlit mucho más rápido

5. Estructura del repositorio

```
OFFICE_MAX_FINAL/
■■■■ app.py                # Aplicación Streamlit: 6 vistas + orquestación + caché
■■■■ requirements.txt      # Dependencias Python
■■■■ README.md            # Este documento (PRD)
■■■■ SPEC_PRICING_ENGINE.md # Brief original de construcción (12 fases)
■■■■ convertir_a_parquet.py # Utilidad CSV/Excel → Parquet
■■
■■■■ .streamlit/
■■■■ ■■■■ config.toml      # Tema oscuro, límites de carga
■■
■■■■ modules/
■■■■ ■■■■ config.py        # Umbrales, escenarios, columnas, pesos, coordenadas
■■■■ ■■■■ utils.py         # Lectura, limpieza, NSE, periodos, KPIs, descargas
■■■■ ■■■■ quality.py       # Diagnóstico de calidad y semáforo
■■■■ ■■■■ elasticity.py    # Motor OLS log-log multi-periodo + NSE (vectorizado)
■■■■ ■■■■ historical_pricing.py # Backtesting de escenarios históricos
■■■■ ■■■■ demand_forecast.py # Proyección de demanda base futura
■■■■ ■■■■ future_pricing.py # Simulación de escenarios futuros
■■■■ ■■■■ promotions.py    # Escenarios y guardrails de promociones
■■■■ ■■■■ recommendations.py # Motor de recomendaciones (reglas + simulación)
■■■■ ■■■■ historical_ml.py  # ML diagnóstico (Logistic Regression + Random Forest)
```

```

■ ■■■ pricing.py # Simulador trimestral legacy (compatibilidad)
■
■ ■■■ tests/ # Pruebas de pricing, demanda, recomendaciones, sintaxis
■
■ ■■■ data/
■ ■■■ inegi/
■ ■■■ catalogo_ubica_geo.csv # Catálogo geográfico key → ubica_geo (1 112 municipios)
■ ■■■ hogares_INEGI.csv # Base NSE por hogar (≈91 k hogares)
■ ■■■ default_nse/
■ ■■■ base_nse_default.csv # NSE default de respaldo (32 municipios)

```

6. Requisitos técnicos

Columnas mínimas en la base de ventas:

Columna	Tipo	Descripción
tran_date	Fecha	Fecha de transacción (dd/mm/YYYY o ISO)
qty	Numérico	Unidades vendidas
net_sale	Numérico	Ingreso neto de la transacción
prod_nbr	Texto	Código de producto (SKU)
costo2	Numérico	Costo unitario del producto

Columnas recomendadas: dept_nm, subdept_nm, marca, store_nm, estado, key, categoria_est_socio.

Librerías principales:

Librería	Uso
streamlit	Interfaz web interactiva
pandas / numpy	Manipulación y cálculo vectorizado
statsmodels	OLS log-log de referencia (p-values, R²)
scipy	Estadístico t para p-values en la ruta vectorizada
scikit-learn	ML diagnóstico (Logistic Regression, Random Forest)
plotly	Visualizaciones interactivas
pyarrow	Lectura rápida de Parquet y CSV
openpyxl	Lectura de Excel

7. Cómo correr localmente

```
cd "C:\ruta\al\proyecto"  
python -m venv .venv  
.\.venv\Scripts\Activate.ps1  
pip install -r requirements.txt  
streamlit run app.py
```

Flujo recomendado: 1. **Vista 1:** subir ventas → Procesar datos 2. **Vista 2:** calcular elasticidad 3. **Vista 3:** calcular pricing histórico (backtesting) 4. **Vista 4:** calcular pricing futuro 5. **Vista 5:** revisar recomendaciones ejecutivas 6. **Vista 6:** descargar tablas de resultados